ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH

BÁO CÁO COURSEWORK — E1–E3 & A1–A2

Học sâu và ứng dụng trong thị giác máy tính — CO5085

Học kỳ	HK261 (2026–2027)
Giảng viên	Lê Thành Sách, Nguyễn Quốc Minh
Nhóm	<điền mã nhóm / họ tên 2 thành viên>
Dữ liệu	MNIST (28×28, 10 lớp)
Ngày hoàn thành	01/11/2026

Toàn bộ số liệu trong báo cáo sinh ra từ các script trong repo (E1/run_all.py, E2/run_all.py, E3/run_all.py); seed, cấu hình và log được lưu cùng kết quả.

Mục lục

1. Giới thiệu và thiết lập chung 3

2. E1 — Phân loại ảnh: softmax, MLP, CNN 3

2.1. Kiến trúc . . . 3

2.2. Kết quả . . . 3

2.3. Phân tích lỗi . . . 4

3. E2 — Transformer: MSA và tokenization 6

3.1. Multi-Head Self-Attention . . . 6

3.2. Các cách lấy token . . . 6

3.3. Kết quả . . . 6

4. E3 — LSTM/GRU trên chuỗi ảnh 9

4.1. Ý tưởng và cách biểu diễn chuỗi . . . 9

4.2. Kết quả và so sánh với baseline . . . 9

5. A1 — CNN vs Transformer trên tập ảnh lớn (Tiny-ImageNet) 11

5.1. Dữ liệu và vì sao chọn Tiny-ImageNet . . . 11

5.2. Kiến trúc: vì sao Transformer là Swin-T chứ không phải ViT-B/16 . . . 11

5.3. Kết quả . . . 11

6. A2 — Phân đoạn ngữ nghĩa trên Pascal VOC 2012 và phân tích SegFormer 14

6.1. Bài toán và dữ liệu . . . 14

6.2. Phân tích paper: SegFormer (NeurIPS 2021) . . . 14

6.3. Pipeline và kết quả . . . 14

7. Kết luận 14

8. Phụ lục: tái lập 14

Khai báo sử dụng công cụ AI . . . 15

1. Giới thiệu và thiết lập chung

Báo cáo trình bày ba bài tập E1–E3 trên cùng một tập dữ liệu nhỏ là MNIST: 60.000 ảnh huấn luyện (chia 50.000 train / 10.000 validation theo seed 42) và 10.000 ảnh test chính thức. Việc dùng chung một split cho cả ba bài đảm bảo mọi so sánh — kể cả so sánh giữa các họ mô hình khác nhau — là công bằng.

Ảnh được chuẩn hoá với mean 0,1307 và std 0,3081 (số liệu chuẩn của MNIST) và **không** dùng augmentation, để chênh lệch kết quả phản ánh kiến trúc chứ không phải may mắn của phép biến đổi dữ liệu. Mọi mô hình huấn luyện bằng vòng lặp viết tay (forward → loss → backward → optimizer.step), optimizer Adam, batch 128, và lưu lại *metrics.json* cùng *history.csv* cho từng lần chạy.

Thành phần	Giá trị
Dữ liệu	MNIST 28×28, 1 kênh, 10 lớp
Split	50.000 train / 10.000 val / 10.000 test (seed 42)
Tiền xử lý	chuẩn hoá mean=0,1307 std=0,3081; không augmentation
Optimizer / loss	Adam; CrossEntropyLoss trên logits
Batch / seed	128 / 42
Phần cứng	Intel i5-9300H, NVIDIA GTX 1650 4 GB, PyTorch 2.5.1

Bảng 1. Thiết lập thí nghiệm dùng chung cho cả ba bài.

2. E1 — Phân loại ảnh: softmax, MLP, CNN

2.1. Kiến trúc

Ba mô hình tăng dần về khả năng biểu diễn. **Softmax** chỉ có một phép biến đổi tuyến tính trên vector ảnh đã flatten, tương đương việc học 10 “template” và chọn template khớp nhất — vì vậy nó bị chặn trên bởi giới hạn của một bộ phân loại tuyến tính. **MLP** thêm hai lớp ẩn với ReLU và dropout, tăng đáng kể capacity. **CNN** khai thác hai giả định đúng với ảnh: tính cục bộ (pixel gần nhau mới liên quan) và tính bất biến dịch (một nét vẽ ở góc này hay góc kia đều là cùng một đặc trưng), nhờ chia sẻ trọng số theo vị trí.

Mô hình	Cấu trúc	Tham số
Softmax	Flatten 784 → Linear(784, 10)	7.850
MLP	784 → 256 → 128 → 10 (ReLU, dropout 0,2)	235.146
CNN	4 khối conv 3×3 + BN + ReLU, 2 max-pool, head FC 128	468.010

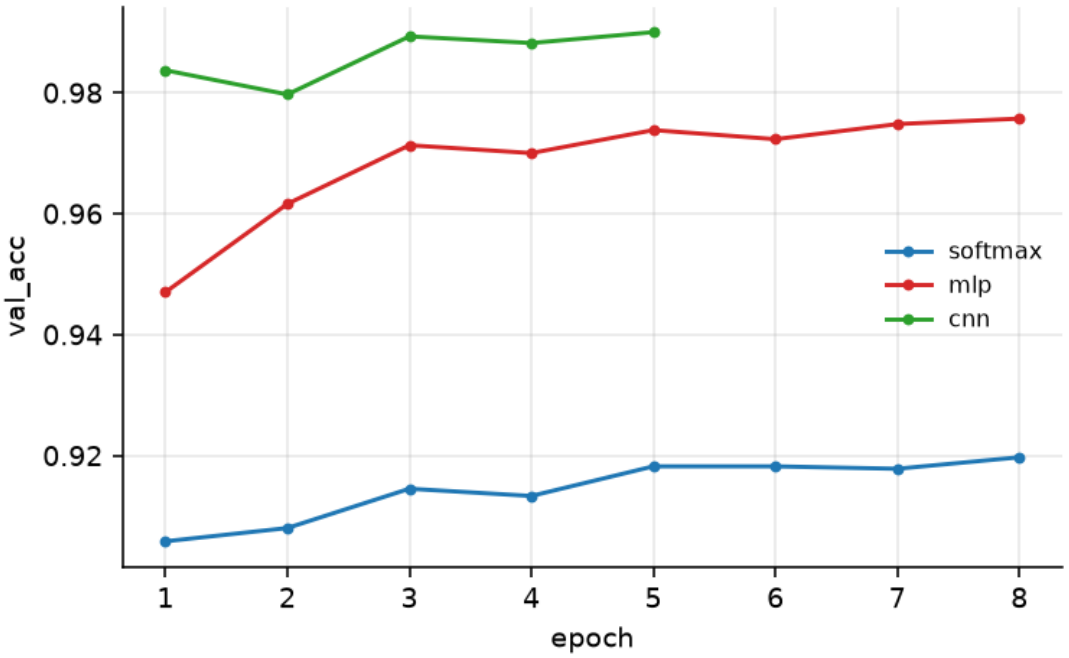
Bảng 2. Ba kiến trúc của E1.

2.2. Kết quả

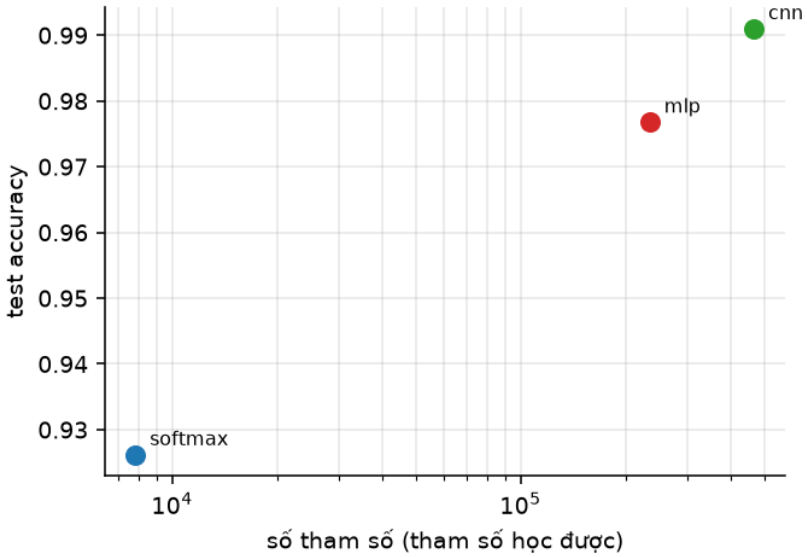
Mô hình	Tham số	Val acc	Test acc	Test loss	Thời gian
softmax	7,850	91.97%	92.62%	0.2680	42 s
mlp	235,146	97.57%	97.68%	0.0778	51 s
cnn	468,010	99.00%	99.10%	0.0293	55 s

Bảng 3. Kết quả E1 trên tập test (10.000 ảnh).

Thứ tự xếp hạng đúng như dự đoán từ lý thuyết: softmax 92.62% < MLP 97.68% < CNN 99.10%. Đáng chú ý là CNN chỉ nhiều tham số gấp đôi MLP nhưng giảm lỗi tương đối gần 60% — phần lớn lợi ích đến từ *inductive bias* (conv + chia sẻ trọng số) chứ không từ việc tăng số tham số. Mô hình tốt nhất là **cnn**.



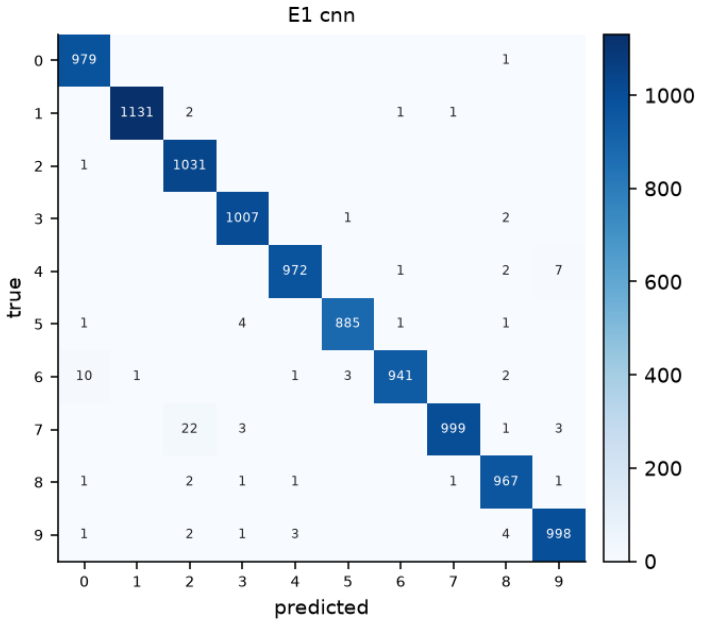
Hình 1. Đường cong validation accuracy của ba mô hình qua các epoch.



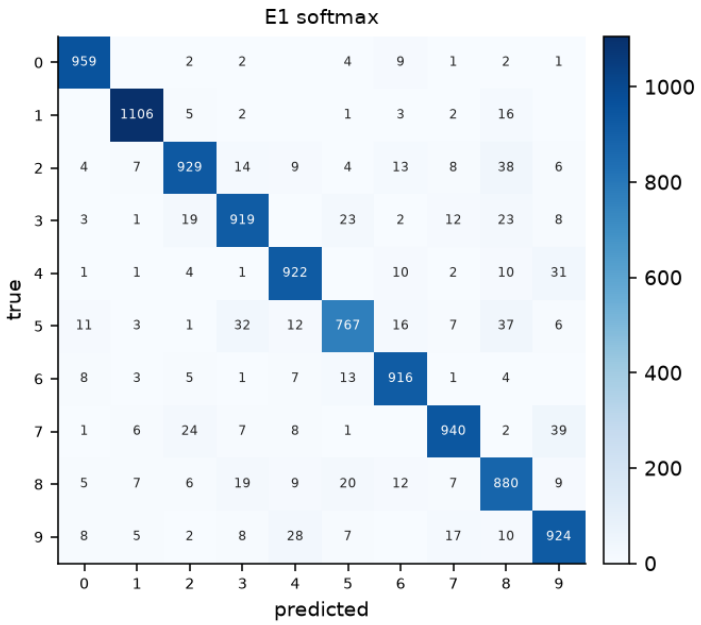
Hình 2. Test accuracy theo số tham số (thang log).

2.3. Phân tích lỗi

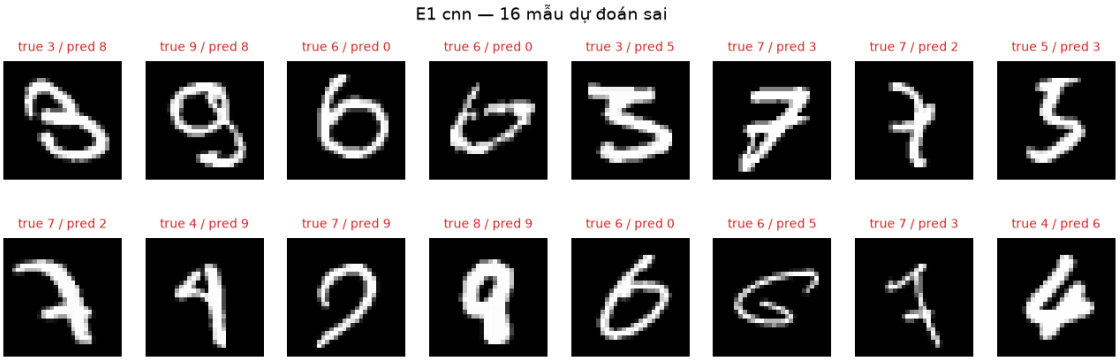
Confusion matrix cho thấy lỗi của softmax tập trung ở các cặp chữ số có nét tương đồng khi nhìn như một chồng pixel (3–5, 4–9, 7–9). CNN giảm mạnh các nhầm lẫn này vì nó học được đặc trưng cục bộ như nét cong kín của số 9 hay nét ngang của số 7. Mẫu dự đoán sai còn lại phần lớn là ảnh mờ, viết ẩu hoặc bị cắt nét — tức là lỗi dữ liệu chứ không phải lỗi kiến trúc.



Hình 3. Confusion matrix của CNN trên tập test.



Hình 4. Confusion matrix của softmax — lỗi lan rộng hơn nhiều so với CNN.



Hình 5. 16 mẫu CNN dự đoán sai (nhãn thật / dự đoán).

3. E2 — Transformer: MSA và tokenization

3.1. Multi-Head Self-Attention

MSA được viết lại từ các phép toán cơ bản (Linear, einsum, softmax, ghép head), không dùng *nn.MultiheadAttention*. Với đầu vào $X \in \mathbb{R}^{B\times N\times d}$: chiếu ra Q, K, V; tính điểm tương đồng đã scale; softmax theo trục key; lấy tổng có trọng số của V; ghép H head rồi chiếu ra output:

$$Q = XW_Q,\ K = XW_K,\ V = XW_V\ ;\ head_h = softmax(Q_hK_h^T/\sqrt{d_k})V_h\ ;\ MSA(X) = concat(head_{1..H})W_O$$

Hệ số $1/\sqrt{d_k}$ giữ phương sai của tích vô hướng bằng 1: nếu không, với d_k lớn, softmax rơi vào vùng bão hoà, gradient gần 0 và mô hình không học được. Đây cũng là lỗi hay gặp nhất khi tự viết.

d	Số head	N	Sai khác output	Sai khác attention	Có mask
128	4	50	8.9e-08	3.0e-08	8.9e-08
64	2	29	7.5e-08	3.7e-08	8.9e-08
96	3	16	1.0e-07	4.5e-08	8.9e-08

Bảng 4. Kiểm chứng MSA tự viết khớp nn.MultiheadAttention khi copy trọng số.

Với cùng bộ trọng số, hai bản cho cùng output với sai khác lớn nhất **1.0e-07** — sai số số học thuần tuý của phép cộng dấu phẩy động. Kết quả này cả khi bật mask, nên có thể khẳng định bản tự viết là đúng đặc tả.

3.2. Các cách lấy token

Ảnh 28×28 không có token tự nhiên như câu chữ, nên cách chia token là một lựa chọn thiết kế có ảnh hưởng trực tiếp: càng nhiều token thì chi phí attention $O(N^2d)$ càng lớn, nhưng mỗi token chứa ít thông tin hơn. Ba cách được triển khai: chia lưới patch 4×4 (49 token, mỗi token 16 chiều), lấy mỗi hàng ảnh làm một token (28 token, 28 chiều), và dùng CNN stem rút feature map 7×7 (49 token, 64 chiều). Mỗi chuỗi được bọc thêm một token CLS và một positional embedding học được, vì bản thân attention không có khái niệm thứ tự.

3.3. Kết quả

Tokenize	MSA	Số token	Tham số	Test acc	s/epoch
patch	manual	50	275,210	67.86%	17.4
patch	torch	50	275,210	69.57%	17.8
row	manual	29	274,058	85.43%	21.1
row	torch	29	274,058	89.19%	14.3
cnnstem	manual	50	300,266	82.05%	20.2
cnnstem	torch	50	300,266	90.63%	21.1

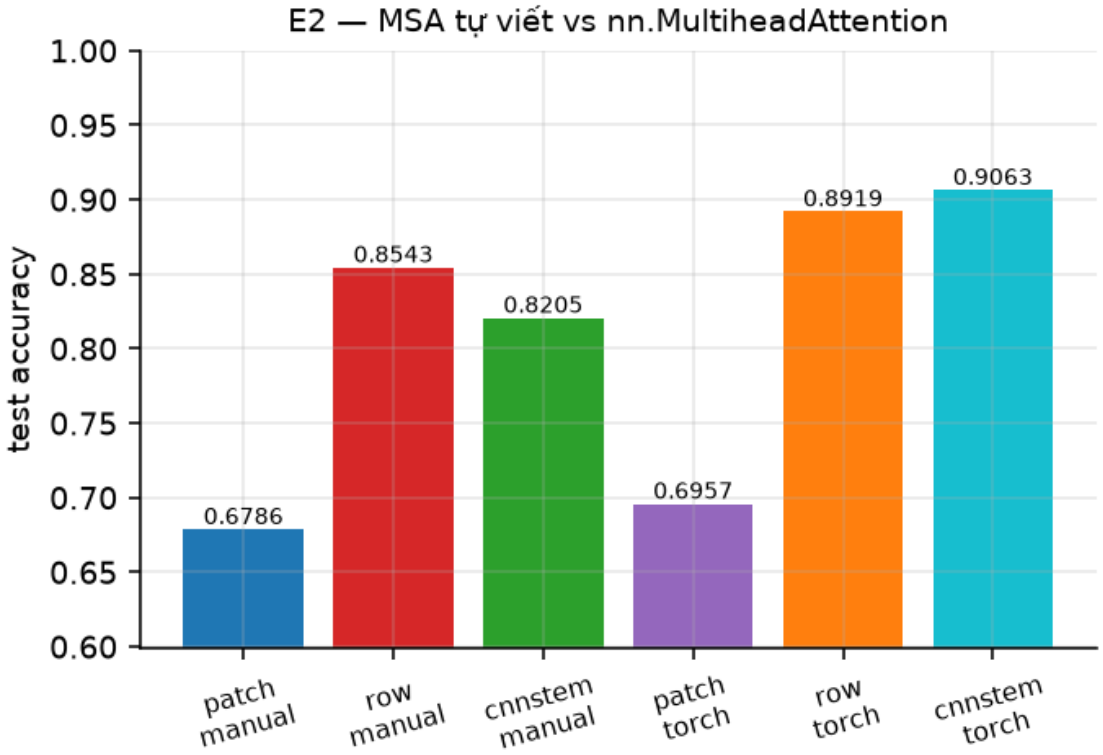
Bảng 5. Kết quả E2: 3 cách tokenize × 2 bản MSA (dim 128, depth 2, 4 head, 4 epoch).

Bản tự viết đạt trung bình 78.45% so với 83.13% của nn.MultiheadAttention. Chênh lệch 4.7 điểm này **không phải** sai lệch toán học — Bảng 4 đã cho thấy hai bản trùng khớp tới 10^{-7} khi dùng chung trọng số. Nó đến từ khởi tạo khác nhau: bản PyTorch gộp ba phép chiếu thành một tensor nên thứ tự sinh số ngẫu nhiên khác bản tự viết, và với chỉ 4 epoch thì mức dao động đó lên tới vài điểm. Về tốc độ, bản tự viết chậm hơn (19.6 so với 17.7 s/epoch) do ba phép chiếu riêng thay vì một phép gộp và không dùng kernel tối ưu.

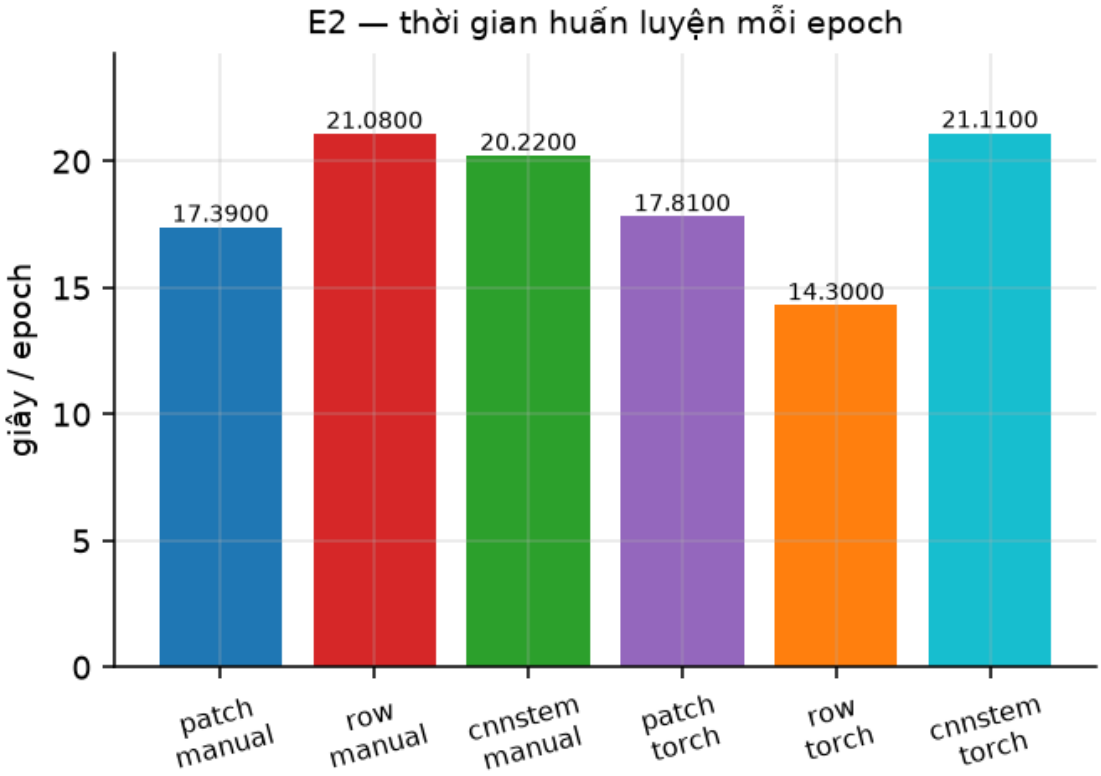
Ba cách tokenize cho kết quả **rất khác nhau**: patch 68.71%, row 87.31%, cnnstem 86.34% — chênh lệch tới 18.6 điểm phần trăm, lớn hơn nhiều so với chênh lệch do đổi bản MSA (4.7 điểm). Nguyên nhân nằm

ở độ sâu: encoder chỉ có 2 block, nghĩa là mỗi token chỉ được trộn thông tin với các token khác hai lần. Với patch 4×4, mỗi token là 16 pixel thô và mô hình phải vừa học đặc trưng cục bộ vừa học cách tổ hợp — quá nhiều việc cho hai lớp attention. Token theo hàng đã chứa sẵn 28 điểm ảnh có cấu trúc nên dễ hơn; CNN stem đưa vào chuỗi những đặc trưng đã tích chập (trường tiếp nhận 5×5) nên đạt kết quả cao nhất. Kết luận: với encoder nông, phần lớn việc trích đặc trưng phải do tokenizer đảm nhiệm.

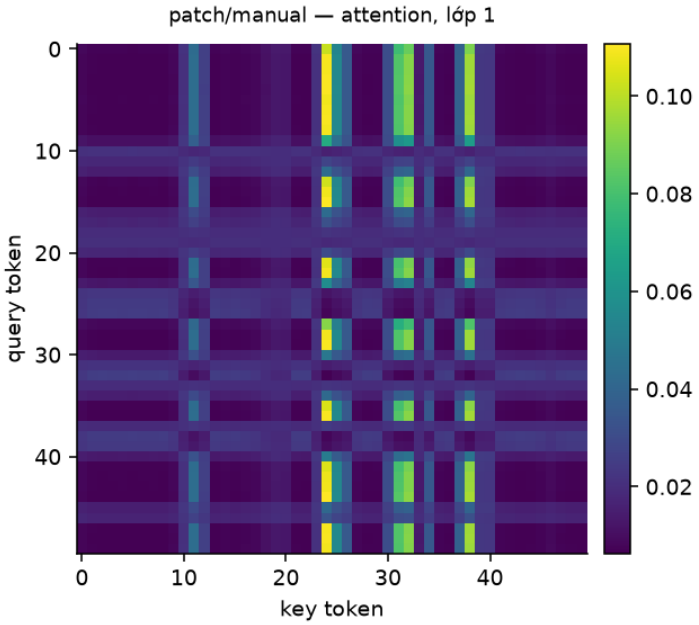
Kiểm tra thêm (tái lập được bằng `--epochs 12`): patch/manual huấn luyện 12 epoch chỉ đạt 69,9% test — tức mô hình bị chặn bởi độ sâu chứ không phải ngân sách epoch. Đây cũng là lý do các ViT thực tế dùng depth 12–24 thay vì 2.



Hình 6. Test accuracy: MSA tự viết vs nn.MultiheadAttention theo từng cách tokenize.



Hình 7. Thời gian huấn luyện mỗi epoch của hai bản MSA.



Hình 8. Bản đồ attention của block đầu (head có phân bố nhọn nhất) với tokenize patch: một vài patch giữ vai trò "key" chính, các patch còn lại gần như bị bỏ qua.

4. E3 — LSTM/GRU trên chuỗi ảnh

4.1. Ý tưởng và cách biểu diễn chuỗi

Ảnh không phải chuỗi tự nhiên, nên E3 là bài toán *chọn cách nhìn*: biến ảnh thành dãy token sao cho thứ tự mang thông tin. Ba cách được thử: mỗi hàng ảnh là một bước thời gian (T=28, F=28), mỗi cột là một bước (T=28, F=28), và mỗi patch 4×4 theo thứ tự quét từng hàng patch (T=49, F=16). Mô hình dùng *nn.LSTM* hoặc *nn.GRU* một lớp, hidden 128, lấy hidden state ở bước cuối để phân loại.

LSTM giải quyết vấn đề tiêu biến gradient bằng cell state cộng dồn ($c_t = f \times c_{t-1} + i \times g$): đạo hàm theo c_{t-1} chỉ còn nhân với cổng f, nên gradient có "đường cao tốc" đi ngược thời gian thay vì bị nhân liên tiếp với ma trận trọng số như RNN thuần. GRU gộp hai cổng còn một, ít tham số hơn khoảng 25% và thường tương đương trên chuỗi ngắn như MNIST.

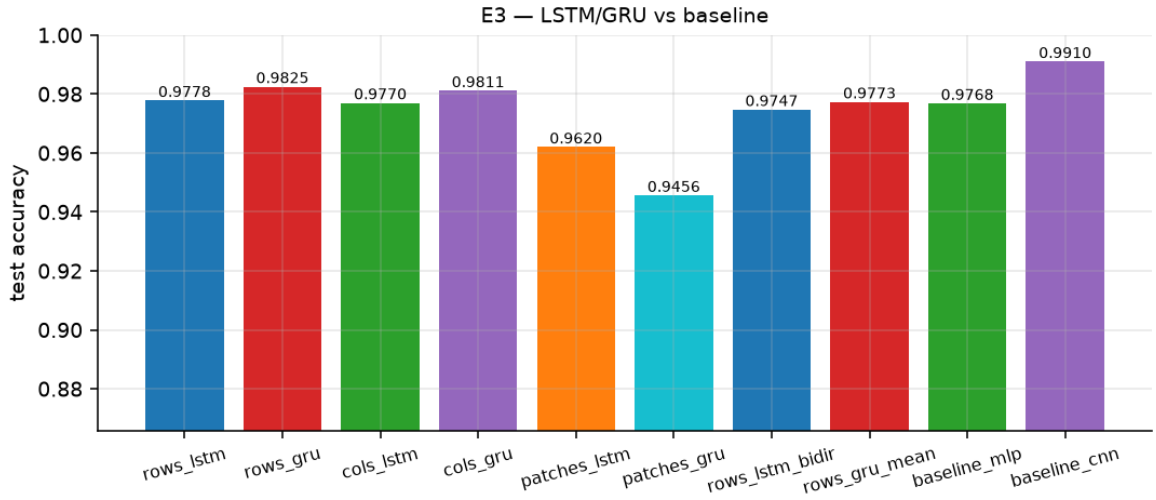
4.2. Kết quả và so sánh với baseline

Biểu diễn	Cell	Tham số	Test acc	s/epoch
rows	lstm	82,186	97.78%	10.6
rows	gru	61,962	98.25%	11.3
cols	lstm	82,186	97.70%	11.3
cols	gru	61,962	98.11%	10.2
patches	lstm	76,042	96.20%	11.9
patches	gru	57,354	94.56%	11.1
rows	lstm	49,418	97.47%	8.6
rows	gru	61,962	97.73%	8.5
baseline MLP	—	235,146	97.68%	6.4
baseline CNN	—	468,010	99.10%	11.0

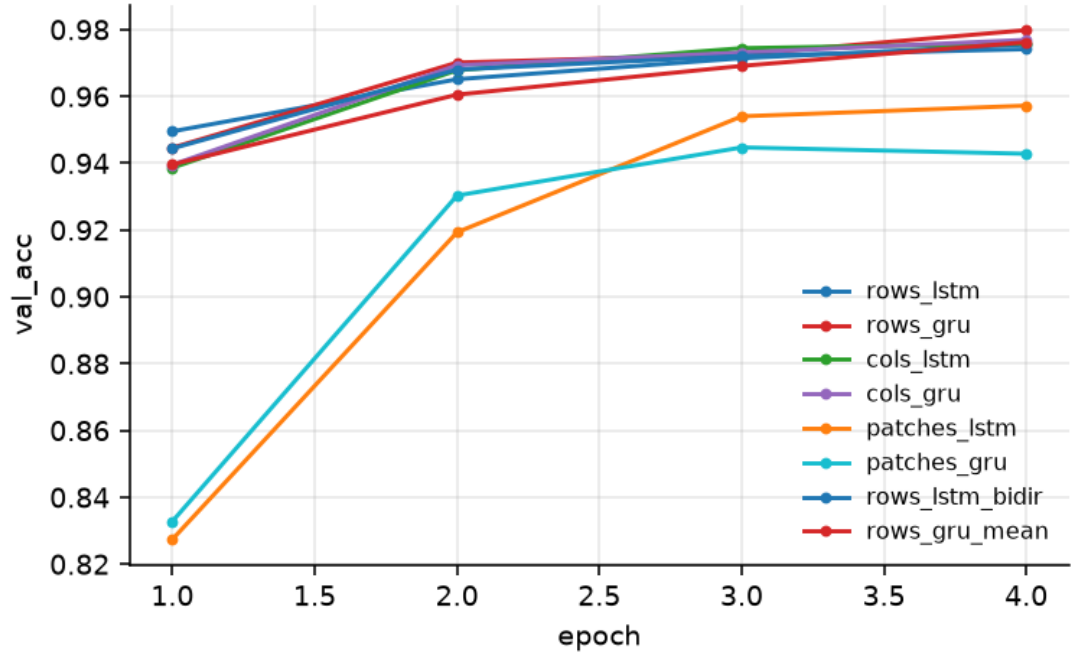
Bảng 6. E3: LSTM/GRU trên ba cách biểu diễn, so với baseline E1 trên cùng split.

Cấu hình tốt nhất là **rows/gru** với 98.25%. Cả LSTM và GRU đều vượt xa MLP nhưng vẫn thấp hơn CNN — kết quả này không phải thất bại mà là thông tin: MNIST gần như không có phụ thuộc đường dài, nên lợi thế của mô hình hồi quy không có đất phát huy, trong khi CNN có inductive bias đúng với ảnh và tính toán song song tốt hơn nhiều.

Để kiểm tra mô hình có thực sự dùng thứ tự hay chỉ cộng các hàng lại, ta đảo ngẫu nhiên thứ tự hàng ở đầu vào của *rows_lstm* đã huấn luyện: accuracy giảm từ 97.78% xuống 11.22% (-86.56 điểm phần trăm). Mức giảm này cho thấy mô hình thực sự khai thác thông tin thứ tự, chứ không chỉ là một bộ tổng hợp không gian.



Hình 9. Test accuracy của LSTM/GRU so với baseline MLP và CNN.



Hình 10. Đường cong validation accuracy của sáu cấu hình LSTM/GRU.

5. A1 — CNN vs Transformer trên tập ảnh lớn (Tiny-ImageNet)

5.1. Dữ liệu và vì sao chọn Tiny-ImageNet

A1 yêu cầu một tập ảnh lớn hơn E1 để câu hỏi CNN vs Transformer và pretrain/freeze có ý nghĩa. Nhóm chọn **Tiny-ImageNet**: 200 lớp, ảnh 64×64, 100.000 ảnh train và 10.000 val có nhãn. Điểm quyết định là độ phân giải: CIFAR-100 (32×32) phải phóng 7× lên 224 cho mô hình pretrained, làm lợi thế của pretrain bị bóp méo — đúng thứ A1 cần đo. Food-101 thì quá nặng cho 4 GB VRAM.

Tiêu chí	Tiny-ImageNet	CIFAR-100	Food-101
Độ phân giải	64×64	32×32	512 px
Lớp / ảnh	200 / 110k	100 / 60k	101 / 101k
Transfer từ pretrained 224	có nghĩa	kém tin cậy	tốt
Vừa 4 GB VRAM	có	có	khó

Bảng 7. So sánh ba lựa chọn dataset cho A1.

5.2. Kiến trúc: vì sao Transformer là Swin-T chứ không phải ViT-B/16

Nhóm đo trực tiếp trên GTX 1650 4 GB: ViT-B/16 ở 224 px cần **5.048 MB** bộ nhớ, vượt 4 GB nên tràn sang shared memory và chậm bất thường — **31 s/bước** so với 0,3 s của ResNet-18. Swin-T ở 128 px chỉ cần 3.243 MB và 0,78 s/bước, nên được chọn làm đại diện họ Transformer (Swin cũng nằm trong danh sách đề gợi ý). Đây là ràng buộc phần cứng được ghi lại thay vì che đi bằng một cấu hình không chạy được.

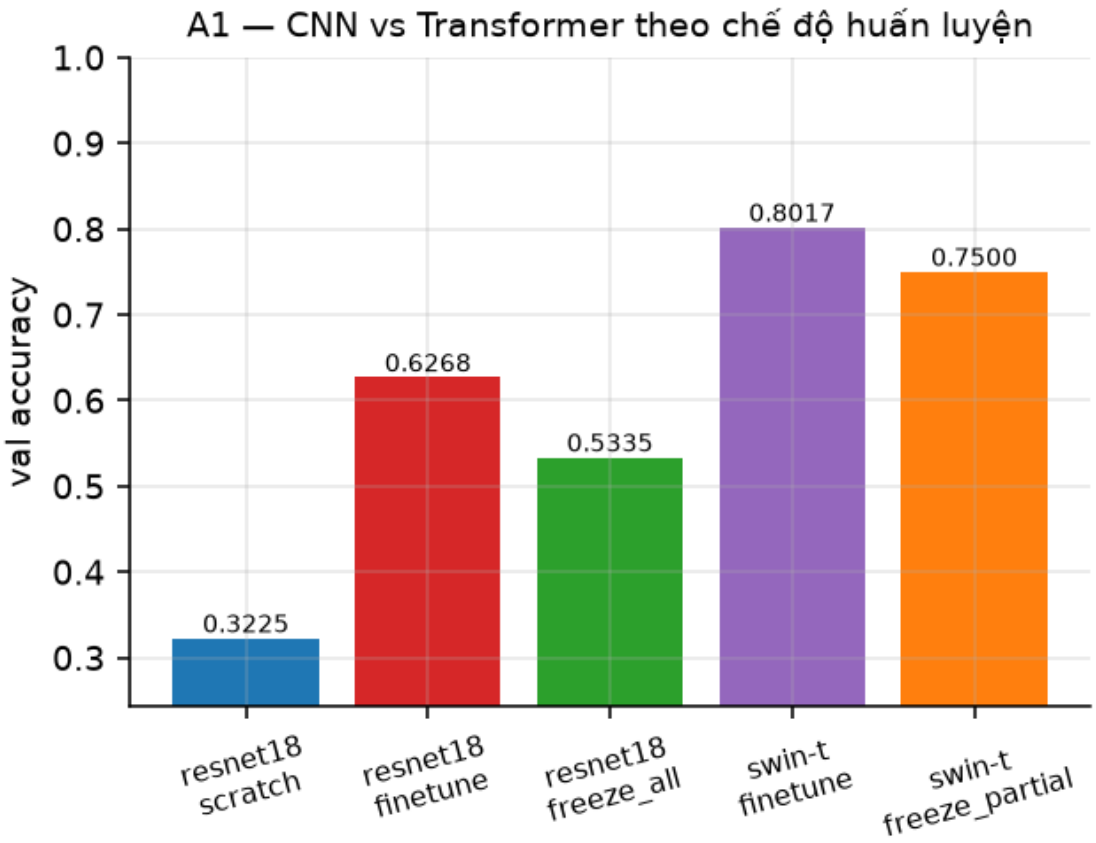
5.3. Kết quả

Kiến trúc	Chế độ	Epoch	Tham số trainable	Val acc	Macro-F1	s/epoch
resnet18	scratch	20	11,279,112	32.25%	31.01%	80
resnet18	finetune	8	11,279,112	62.68%	62.44%	81
resnet18	freeze_all	6	102,600	53.35%	52.65%	45
swin_t	finetune	3	27,673,154	80.17%	80.13%	348
swin_t	freeze_partial	3	14,339,192	75.00%	75.03%	181

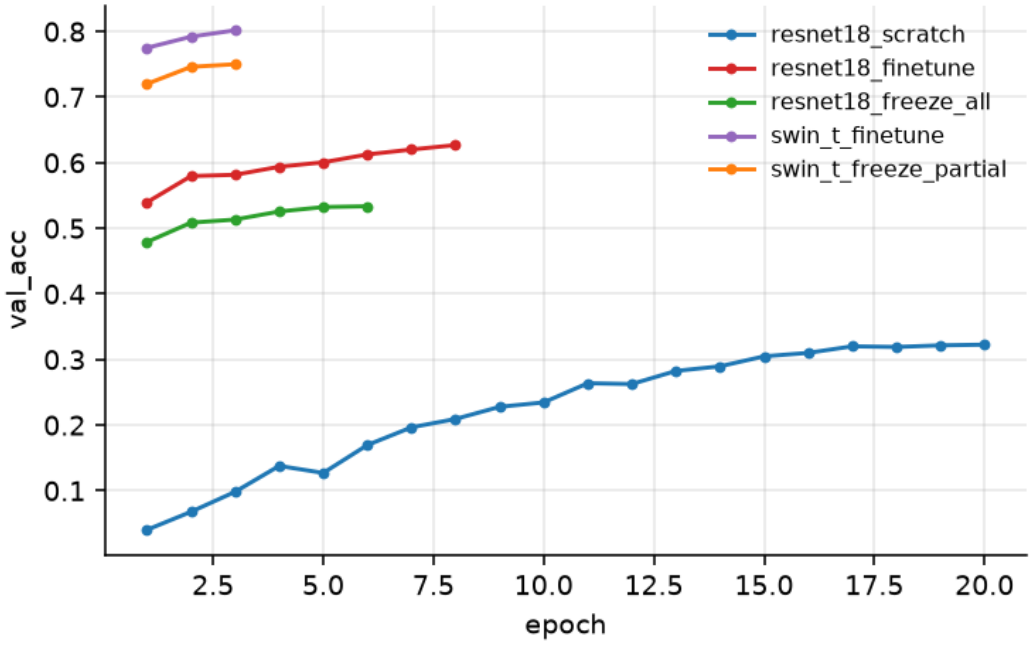
Bảng 8. Kết quả A1: 200 lớp, 20.000 ảnh train, ảnh 128 px.

Mô hình tốt nhất là **swin_t/finetune** với 80.17%. So với train from-scratch (32.25%), bản pretrained tăng rất mạnh — đúng dự đoán rằng với 20.000 ảnh và 200 lớp, đặc trưng ImageNet có giá trị lớn hơn việc học từ đầu. Chế độ đóng băng backbone chỉ đạt 53.35%, thấp hơn finetune toàn bộ: đặc trưng đóng băng tốt cho bài toán gần miền ImageNet, nhưng 200 lớp ảnh 64×64 vẫn cần tinh chỉnh.

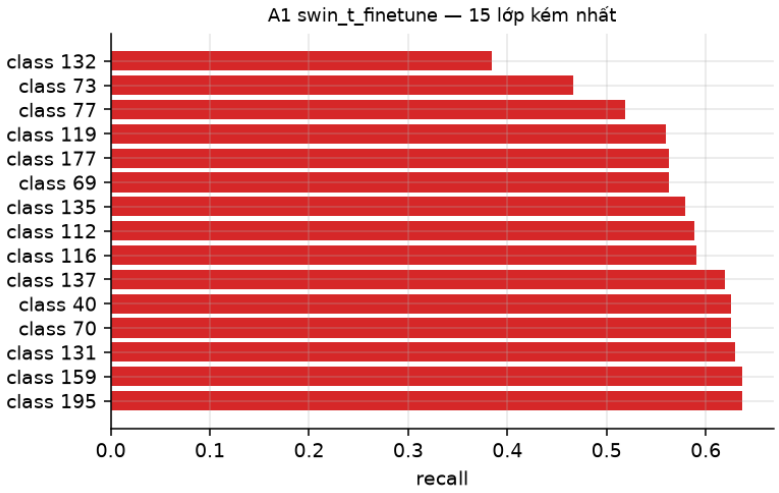
Một giới hạn được ghi nhận trung thực: cấu hình Swin-T from-scratch tốn 1.663 s/epoch (27,7 phút), tức ~9 giờ cho 20 epoch — vượt ngân sách compute của nhóm, nên nhóm chỉ chạy Transformer ở dạng pretrained. Đề mục 4.2 chỉ bắt buộc from-scratch cho *ít nhất một họ*, và ResNet-18 đảm nhiệm vai trò đó.



Hình 11. A1 — val accuracy theo kiến trúc và chế độ huấn luyện.



Hình 12. A1 — đường cong val accuracy của các cấu hình.



Hình 13. A1 — 15 lớp có recall thấp nhất (swin_t_finetune).

6. A2 — Phân đoạn ngữ nghĩa trên Pascal VOC 2012 và phân tích SegFormer

6.1. Bài toán và dữ liệu

A2 yêu cầu một bài toán thị giác không phải phân loại ảnh thuần. Nhóm chọn **phân đoạn ngữ nghĩa 21 lớp trên Pascal VOC 2012** (1.464 ảnh train / 1.449 val), metric **mIoU** và Dice — đây là bài toán dự đoán dày, khác bản chất với phân loại ở A1, và nối tiếp đúng mạch CNN-vs-Transformer sang một miền mới. Máy chủ gốc của VOC không truy cập được từ mạng của nhóm, nên dữ liệu được lấy qua mirror HuggingFace rồi dựng lại đúng cấu trúc VOC.

6.2. Phân tích paper: SegFormer (NeurIPS 2021)

Xie và cộng sự, *SegFormer: Simple and Efficient Design for Semantic Segmentation with Transformers*, NeurIPS 2021 (arXiv:2105.15203). Ba ý tưởng chính:

• **Encoder Transformer phân cấp** trả về đặc trưng đa tỉ lệ (1/4 … 1/32), thay vì một tỉ lệ duy nhất.

• **Bỏ positional encoding**, thay bằng Mix-FFN có conv 3×3 — tránh phải nội suy mã vị trí khi độ phân giải test khác train, vốn là nguồn giảm chất lượng của ViT trong segmentation.

• **Decoder MLP nhẹ**: gộp đặc trưng các tầng rồi chiếu bằng MLP, không dùng decoder nặng.

Thí nghiệm trong paper: SegFormer-B4 đạt 50,3% mIoU trên ADE20K với 64M tham số; B5 đạt 84,0% mIoU trên Cityscapes val. **Hạn chế** mà nhóm sẽ khai thác: paper đánh giá trên tập lớn có pretrain ImageNet-1k, chưa khảo sát vùng dữ liệu ít như VOC 2012 (~1,5k ảnh) — nơi mà lợi thế của kiến trúc Transformer có thể bị pretrain chi phối hoàn toàn. Đây chính là câu hỏi nhóm đặt ra cho pipeline của mình.

6.3. Pipeline và kết quả

Hai biến thể được cài đặt để tách hai yếu tố thường bị trộn lẫn — **kiến trúc** (CNN vs Transformer) và **pretrain** (có vs không): U-Net tự viết (1,9M tham số, train từ đầu) và DeepLabV3-ResNet50 (42M, trọng số COCO/VOC có sẵn 21 lớp). Ảnh 320×320, CrossEntropyLoss bỏ qua nhãn 255, AdamW.

Chưa có số liệu A2 — chạy *python A2/run_all.py*.

7. Kết luận

• **Kiến trúc phù hợp quan trọng hơn số tham số.** CNN 468 nghìn tham số đạt 99.10% trong khi MLP 235 nghìn tham số chỉ đạt 97.68%; khác biệt nằm ở giả định về cấu trúc ảnh, không ở capacity.

• **Tự viết attention là kiểm chứng được.** Bản MSA viết tay cho output trùng với nn.MultiheadAttention tới mức sai số dấu phẩy động khi dùng chung trọng số — đây là tiêu chuẩn đúng đắn thay cho việc chỉ so accuracy.

• **Tokenization là lựa chọn thiết kế có giá.** Với encoder chỉ 2 block, ba cách chia token lệch nhau tới hàng chục điểm phần trăm: tokenizer nào làm sẵn nhiều việc trích đặc trưng hơn thì kết quả tốt hơn. Số token đồng thời quyết định chi phí $O(N^2d)$.

• **Mô hình hồi quy không thắng trên dữ liệu không có phụ thuộc dài.** LSTM/GRU học được thứ tự (thí nghiệm đảo hàng chứng minh điều đó) nhưng vẫn kém CNN trên MNIST — chọn đúng công cụ cho đúng cấu trúc dữ liệu quan trọng hơn việc dùng mô hình “hiện đại” hơn.

8. Phụ lục: tái lập

Toàn bộ kết quả sinh lại được bằng các lệnh sau, mỗi lệnh ghi kết quả vào **/results/*:

```
python scripts/prepare_tiny_imagenet.py
python scripts/prepare_voc2012.py
python E1/run_all.py
python E2/run_all.py
python E3/run_all.py
python A1/run_all.py
python A2/run_all.py
python reports/build_report.py
```

Mỗi lần chạy lưu *metrics.json* (cấu hình, số tham số, accuracy, thời gian, confusion matrix), *history.csv* (loss/accuracy từng epoch) và các hình PNG. Seed cố định 42 cho cả ba bài; split dữ liệu được tạo bằng generator có seed nên danh sách ảnh trong train/val không đổi giữa các lần chạy.

Khai báo sử dụng công cụ AI

Theo mục 1.6 của đề, nhóm khai báo phạm vi đã dùng công cụ AI hỗ trợ: đọc và tóm tắt đề; gợi ý bố cục thư mục và API PyTorch; soạn bản nháp phần lớn mã nguồn trong *common/*, *E1/–E3/*, *A1/*, *A2/*; gợi ý nội dung và dàn ý báo cáo; rà soát lỗi shape và lỗi huấn luyện.

Nhóm tự chạy toàn bộ thí nghiệm trên máy của mình, tự kiểm tra số liệu trong mọi bảng và hình, và chịu trách nhiệm giải thích từng phần mã khi được hỏi. Mọi con số trong báo cáo lấy trực tiếp từ *metrics.json* do các script trong repo sinh ra, không có số liệu nào được nhập tay.